Лекция №11. Методология моделирования предметной области

1. Структурная модель предметной области.

2. Функционально-ориентированные и объектно-ориентированные

методологии описания предметной области.

3. Синтетическая методика.

Исторически первым был структурный подход, который возник в конце 60-х вместе с появлением методологии SADT (Structured Analysis and Design Technique), которая позже вошла в набор стандартов IDEF (Icam DEFinition). Набор этих стандартов был разработан в рамках программы ICAM (Integrated Computer Automated Manufacturing), проводимой по инициативе ВВС США.

Прежде чем строить информационную систему, нужно понять, как устроена та область, для которой эта система создаётся. Нельзя написать программу для склада, не разобравшись, что такое товар, поставка, накладная и как они связаны между собой. Именно для этого строится **модель предметной области**.

Модель предметной области — это система, которая имитирует структуру или функционирование реальной предметной области и отвечает главному требованию: быть **адекватной** этой области. То есть модель должна отражать то, что есть на самом деле, а не то, что хотелось бы разработчику.

Зачем это нужно? Без предварительного моделирования велика вероятность допустить ошибки в стратегических вопросах: неверно понять, какие данные хранить, какие функции реализовать, как связать подсистемы. А исправлять такие ошибки на поздних этапах дорого — иногда дороже, чем всю систему переписать заново. Поэтому все современные технологии проектирования ИС начинаются именно с моделирования.

К моделям предъявляют четыре основных требования:

Формализация. Модель должна однозначно описывать структуру предметной области. Не «примерно так», а точно: вот эти объекты, вот эти связи, вот эти правила.

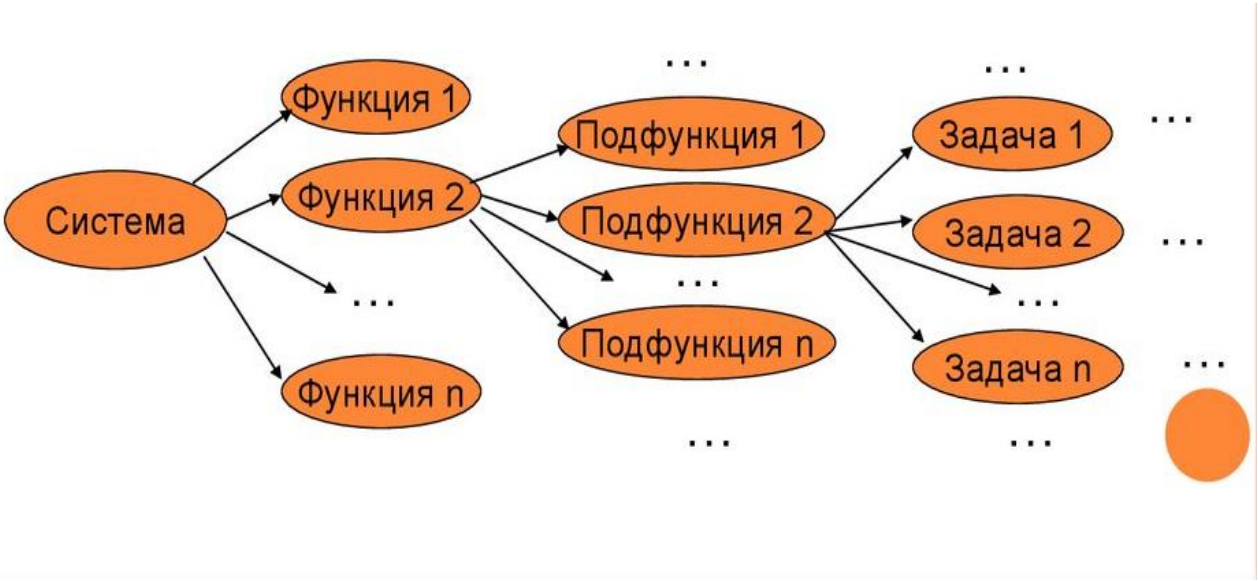
Понятность. Модель должна быть понятна и заказчику, и разработчику. Поэтому используют графические средства — схемы, диаграммы, а не многостраничные тексты.

Реализуемость. Должны существовать средства, с помощью которых модель можно превратить в работающую систему.

Оцениваемость. Должны быть методы и показатели, позволяющие оценить, насколько эффективно модель реализована.

Структурный подход к проектированию информационных систем (ИС) — это методология, основанная на принципе декомпозиции: сложную систему разбивают на иерархически упорядоченные, взаимосвязанные подсистемы, подфункции и задачи,

вплоть до конкретных процедур обработки данных. При этом сохраняется целостное представление о системе, где все компоненты взаимоувязаны.



Основные принципы

Структурный подход базируется на ряде ключевых принципов:

- **«Разделяй и властвуй»** — решение сложной задачи путём разбиения её на множество меньших независимых подзадач, которые проще понять и решить.
- **Иерархическое упорядочивание** — организация подсистем и задач в древовидные структуры с постепенным добавлением деталей на каждом уровне.
- **Абстрагирование** — выделение существенных аспектов системы и отвлечение от несущественных для упрощения её представления.
- **Формализация** — строго методический подход к решению задачи.
- **Упрятывание (инкапсуляция)** — каждый элемент системы на конкретном этапе «знает» только необходимую ему информацию.
- **Концептуальная общность** — следование единой философии на всех этапах жизненного цикла ИС (от анализа до тестирования).
- **Полнота** — контроль на отсутствие лишних элементов.
- **Непротиворечивость** — обоснованность и согласованность всех элементов модели.
- **Логическая независимость** — концентрация на логическом проектировании, а не на физическом.
- **Структурирование данных** — организация данных в иерархические структуры.

Методологии и инструменты

Для реализации структурного подхода используются специализированные методологии и нотации:

- **SADT (Structured Analysis and Design Technique)** — методология структурного анализа и проектирования, известная также как стандарт IDEF0.
- **DFD (Data Flow Diagrams)** — диаграммы потоков данных, описывающие взаимодействие источников и потребителей информации через процессы системы.
- **ERD (Entity-Relationship Diagrams)** — диаграммы «сущность-связь», используемые для моделирования структуры базы данных.
- **STD (State Transition Diagrams)** — диаграммы переходов состояний, характеризующие поведение системы во времени.

SADT представляет собой совокупность методов, правил и процедур, предназначенных для построения функциональной модели объекта какой-либо предметной области. Функциональная модель отображает функциональную структуру объекта, то есть производимые им действия и связи между этими действиями .

Правила SADT включают :

- ограничение количества блоков на каждом уровне декомпозиции (правило 3–6 блоков);
- связность диаграмм (нумерация блоков);
- уникальность меток и наименований;
- синтаксические правила для графики;
- разделение входов и управлений;
- отделение организации от функции.

Семейство IDEF

На основе SADT была разработана методология **IDEF** (ICAM DEFinition), которая стала основной частью программы ICAM (Интеграция компьютерных и промышленных технологий), проводимой по инициативе ВВС США .

IDEF — это семейство методов, каждый из которых предназначен для моделирования определённого аспекта системы. Всего существует 16 методов (от IDEF0 до IDEF14, включая IDEF1X) .

Основные методы IDEF:

Метод	Назначение
IDEF0	Функциональное моделирование
IDEF1	Информационное моделирование

Метод	Назначение
IDEF1X	Моделирование данных
IDEF2	Имитационное моделирование
IDEF3	Описание процессов
IDEF4	Объектно-ориентированное проектирование
IDEF5	Описание онтологий

Наиболее часто используются **IDEF0** и **IDEF3** .

Нотация IDEF0: функциональное моделирование

IDEF0 предназначена для описания функций системы и процессов, которые в ней происходят. Модель представляет собой иерархию взаимосвязанных функций .

Основные элементы IDEF0:

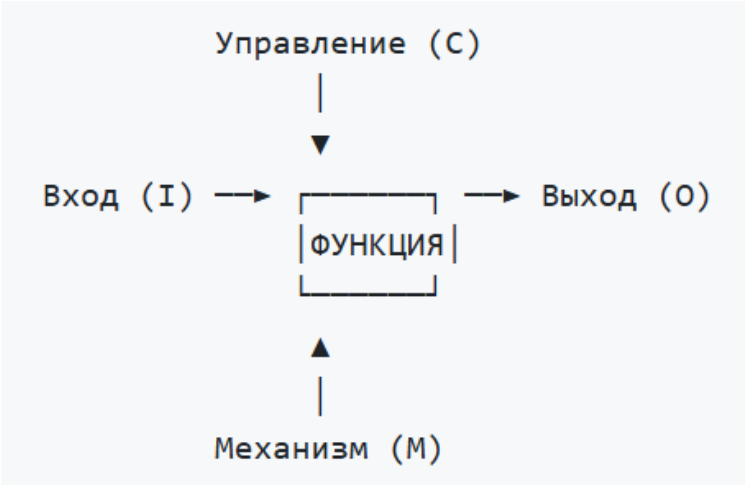
Функциональный блок — прямоугольник, обозначающий функцию. Название блока — глагол или отглагольное существительное (например, «Обработать заказ») .

Стрелки (дуги) — линии со стрелками, показывающие взаимодействие. Существует четыре типа стрелок, которые обозначаются аббревиатурой **ICOM** :

- **I (Input)** — вход. То, что потребляется в ходе выполнения процесса. Стрелка входит в блок слева.
- **C (Control)** — управление. Ограничения и инструкции, влияющие на ход процесса. Стрелка входит в блок сверху.
- **O (Output)** — выход. Результат выполнения процесса. Стрелка выходит из блока справа.
- **M (Mechanism)** — механизм. То, что используется для выполнения процесса (люди, оборудование), но не потребляется. Стрелка входит в блок снизу.

Каждый блок обязательно имеет хотя бы одну управляющую и одну исходящую дугу, поскольку каждый процесс должен происходить по правилам и давать результат .

Иерархия диаграмм. Модель IDEF0 строится по принципу «снаружи вовнутрь». Сначала создаётся **контекстная диаграмма** — описание системы в целом. Затем каждый блок может быть **декомпозирован** на подблоки, которые раскрывают его содержание более детально. Дочерняя диаграмма должна быть согласована с родительской: все граничные стрелки должны продолжаться.



Запомнить легко по аббревиатуре ICOM: Input, Control, Output, Mechanism.

Пример: «Приготовить кофе»

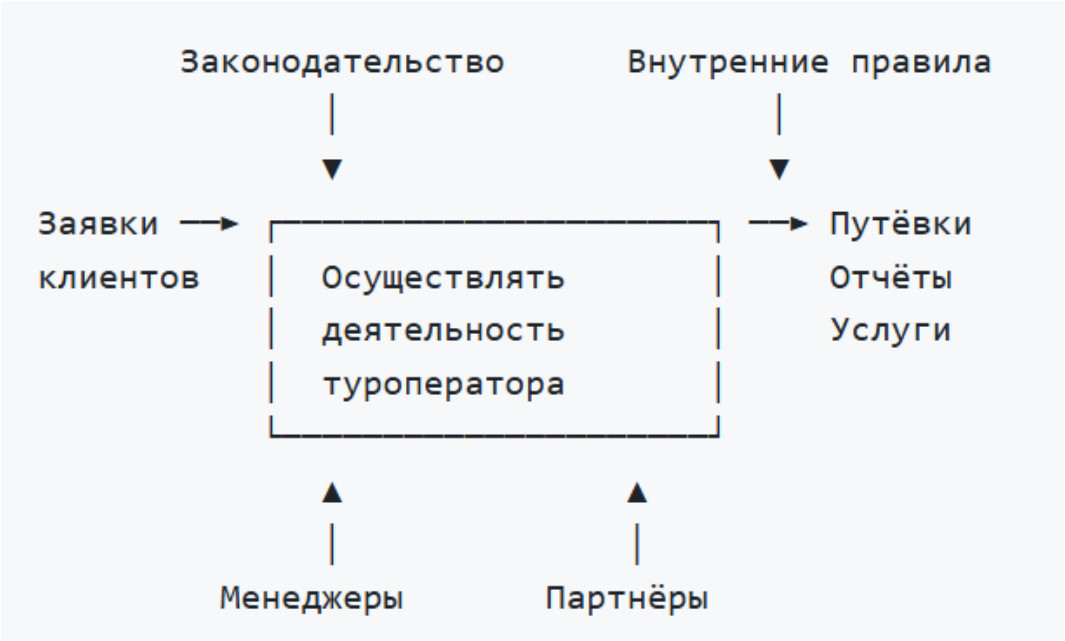
Тип	Что это
Вход	Вода, кофе, чашка
Управление	Рецепт, инструкция кофемашины
Выход	Готовый кофе
Механизм	Кофемашина, человек

Согласитесь, это понятнее, чем просто «приготовить кофе». Здесь видно, что потребляется, чем регулируется, что получается и чем делается.

Контекстная диаграмма

Любая модель IDEF0 начинается с контекстной диаграммы (уровень A-0). Это взгляд на систему как на один большой блок.

Например, для туроператора:



Всё — в одном блоке. Видны внешние связи: что приходит, чем управляется, что выходит, что используется.

Декомпозиция

Потом этот единственный блок разбивается на 3–6 подфункций. Это уровень A0.

Для туроператора:

A1: Формировать туры

A2: Работать с клиентами

A3: Взаимодействовать с партнёрами

A4: Вести отчётность

Каждый блок на этой диаграмме — часть общей функции. Стрелки, которые были на контекстной диаграмме, распределяются между блоками.

Затем каждый блок можно разбить дальше — например, A2 «Работать с клиентами» на:

- A21: Принять заявку
- A22: Подобрать тур
- A23: Оформить путёвку
- A24: Сопровождать клиента

И так далее — пока не станет достаточно детально.

Правила IDEF0

Правило 3–6 блоков. Меньше трёх — диаграмма неглубокая. Больше шести — нечитаемая. Идеально — четыре-пять.

Нумерация блоков. Сверху слева — вниз направо. Первый блок — A1, второй — A2 и так далее.

Граничные стрелки. Если стрелка была на родительской диаграмме, она должна быть и на дочерней. Связь не теряется.

У каждого блока — хотя бы одна стрелка управления и один выход. Без управления функция не работает. Без выхода — зачем она вообще?

Зачем это нужно

Диаграмма IDEF0 — это не картинка ради картинки. Она помогает:

- Договориться. Заказчик и разработчик смотрят на одну схему и видят одно и то же.
- Найти узкие места. Видно, где функция перегружена, где не хватает данных, где нет управления.
- Разделить работу. Каждый блок можно отдать отдельному разработчику.
- Задокументировать. Когда через год придётся что-то менять, схема объяснит, как система устроена.

IDEF3: описание процессов

Если IDEF0 отвечает на вопрос **«что делает система»**, то IDEF3 — на вопрос **«как именно протекает процесс»**.

Представьте, что IDEF0 — это крупный план здания. IDEF3 — это маршрут, по которому идёт человек внутри.

Основные элементы

Единица поведения (UOB) — прямоугольник с названием действия. Например, «Проверить документы», «Отправить заявку».

Связи — стрелки между UOB. Три типа:

- **Предшествования** — показывают, что одно действие идёт после другого.
- **Отношения** — показывают логическую связь (не обязательно временную).
- **Потока объектов** — показывают, как объект переходит из одного действия в другое.

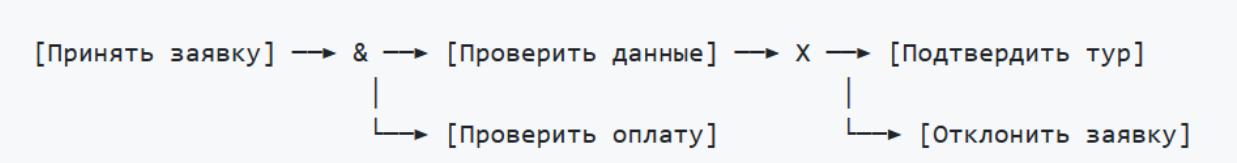
Перекрёстки (Junctions) — элементы, где поток разветвляется или сливается.

Три типа:

Символ	Название	Что означает
&	AND	Все ветви выполняются
O	OR	Хотя бы одна ветвь

Символ	Название	Что означает
X	XOR	Только одна ветвь

Пример: обработка заявки клиента



Здесь & означает, что обе проверки выполняются параллельно. X означает, что после проверок либо тур подтверждается, либо отклоняется — не оба действия сразу.

Зачем это нужно

IDEF3 используется, когда нужно показать:

- Последовательность действий.
- Условия перехода от одного действия к другому.
- Параллельные процессы.
- Точки принятия решений.

Часто IDEF3 дополняет IDEF0: сначала описываем функции, потом раскрываем, как каждая из них выполняется пошагово.

Как это применяется на практике

Анализ предметной области. Прежде чем писать код, аналитик строит модель IDEF0: что делает система, что поступает на вход, чем управляется, что выходит.

Реинжиниринг бизнес-процессов. Компания хочет улучшить работу. Строится модель «как есть» (AS-IS), анализируются узкие места, строится модель «как будет» (TO-BE).

Проектирование ИС. На основе IDEF0 выделяются модули системы. На основе IDEF3 — сценарии работы пользователя.

Документирование. Модели IDEF служат документацией: их читают новые сотрудники, по ним проверяют соответствие системы требованиям.

Ограничения структурного подхода

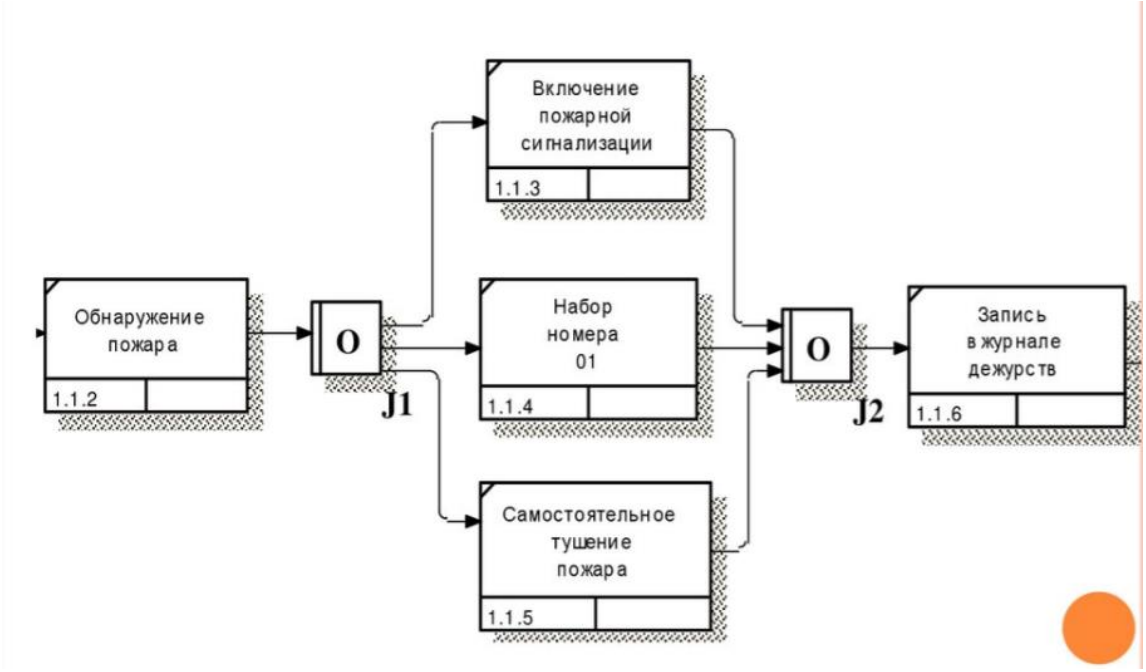
Структурный подход хорош для **статических** описаний: функции, данные, структура.

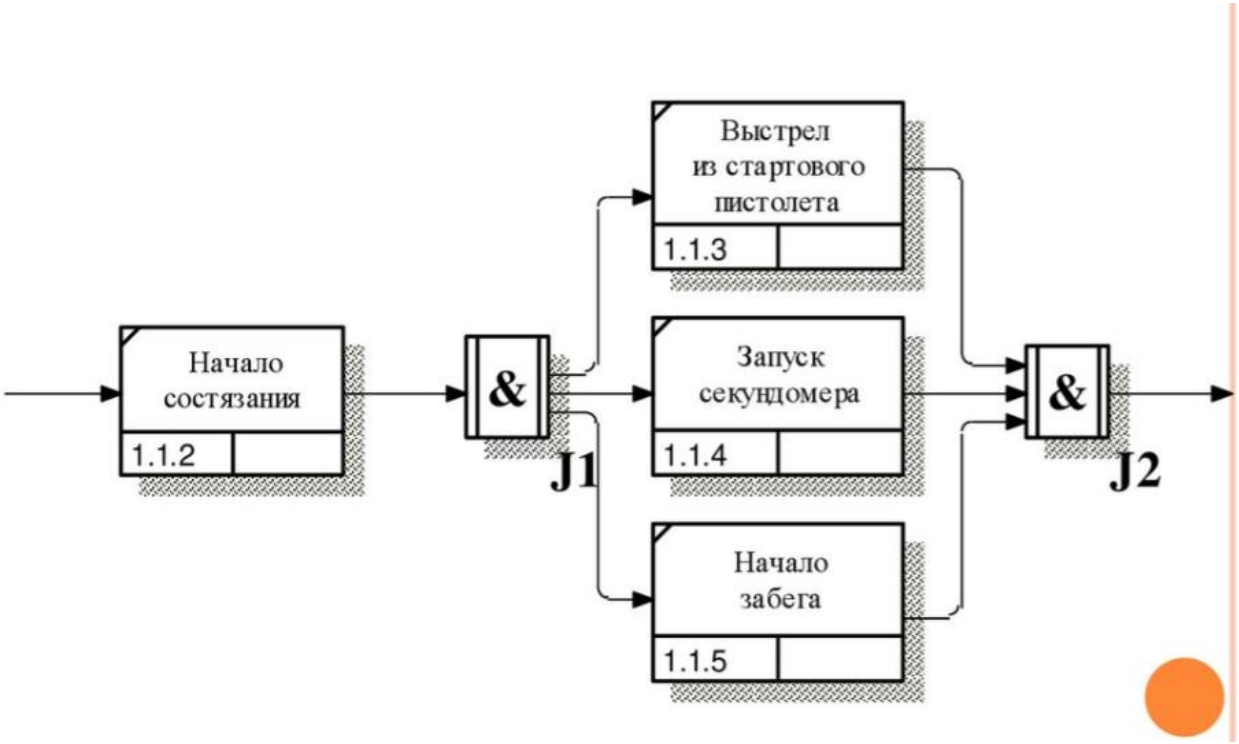
Но он плохо показывает **динамику**: изменение состояний, события, реакцию на внешние воздействия. Поэтому в современных проектах IDEF0 и IDEF3 часто

дополняются диаграммами состояний (State Machine) и диаграммами последовательностей (Sequence Diagram).

Тем не менее, IDEF остаётся стандартом де-факто для описания функциональности системы. На собеседованиях аналитиков часто спрашивают: «Умеете строить IDEF0?» — и это не случайно.

- **Структурный подход** — это способ разбить сложную систему на простые части и описать связи между ними.
- **SADT** — первая методология, на основе которой возникло семейство **IDEF**.
- **IDEF0** описывает функции системы через блоки и стрелки ICOM: Input, Control, Output, Mechanism.
- **IDEF3** описывает процессы: последовательность действий, перекрёстки, условия.
- Модели строятся иерархически: от контекстной диаграммы до детальных уровней.
- IDEF0 отвечает на вопрос «что?», IDEF3 — на вопрос «как?».
- Эти нотации помогают договориться, найти узкие места, разделить работу и задокументировать систему.





Методология DFD

DFD (Data Flow Diagram) — методология построения модели системы в виде диаграммы потоков данных. Основное средство моделирования функциональных требований к проектируемой системе.

Основные понятия DFD:

Потоки данных — абстракции для моделирования передачи информации из одной части системы в другую. Изображаются именованными стрелками.

Процесс (работа) — преобразование входных потоков в выходные в соответствии с действием, задаваемым именем процесса. Имя должно содержать глагол в неопределённой форме, например «Получить документы по отгрузке продукции». У каждого процесса есть уникальный номер.

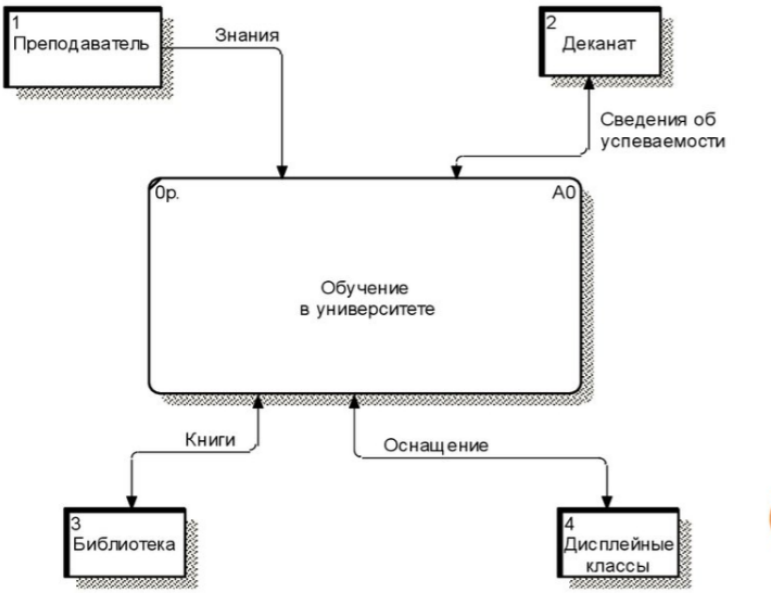
Хранилище (накопитель) данных — позволяет определять данные, которые будут сохраняться в памяти между процессами. Фактически это «срезы» потоков данных во времени. Имя хранилища должно определять его содержание.

Внешняя сущность — материальный объект вне контекста системы, являющийся источником или приёмником системных данных. Например, «склад товаров». Предполагается, что внешние сущности не участвуют в обработке.

Кроме основных элементов, в состав DFD входят **словари данных** (каталоги всех элементов данных) и **миниспецификации** (описание алгоритмов преобразования входных данных в выходные).

Построение иерархии DFD

1. Построение диаграмм уровня системы и подсистемы



2. Построение диаграмм уровня процесса



Преимущества DFD:

- Возможность однозначно определить внешние сущности, анализируя потоки информации внутри и вне системы.
- Проектирование «сверху вниз», что облегчает построение модели «как должно быть».
- Наличие спецификаций процессов нижнего уровня, позволяющих преодолеть логическую незавершённость функциональной модели.

Недостатки DFD:

- Необходимость искусственного ввода управляющих процессов.
- Отсутствие понятия времени, то есть отсутствие анализа временных промежутков при преобразовании данных.

Объектно-ориентированная методология

Принципиальное отличие объектного подхода от функционального — в **способе декомпозиции системы**. Объектно-ориентированный подход использует объектную декомпозицию: статическая структура описывается в терминах объектов и связей между ними, а поведение системы — в терминах обмена сообщениями между объектами.

Основные понятия:

Объект — предмет или явление, имеющее чётко определённое поведение и обладающее состоянием, поведением и индивидуальностью.

Класс — множество объектов, связанных общностью структуры и поведения.

Инкапсуляция — объединение данных и методов их обработки в одном объекте.

Наследование — построение новых классов на основе существующих с возможностью добавления или переопределения данных и методов.

Полиморфизм — способность класса принадлежать более чем одному типу.

Концептуальная основа объектно-ориентированного подхода строится с учётом принципов: абстрагирование, инкапсуляция, модульность, иерархия, типизация, параллелизм, устойчивость.

Важное качество объектного подхода — согласованность моделей деятельности организации и моделей проектируемой ИС от стадии формирования требований до стадии реализации. По объектным моделям может быть прослежено отображение реальных сущностей моделируемой предметной области в объекты и классы информационной системы.

В качестве языка моделирования объектного подхода используется **UML** (Unified Modeling Language) — язык для определения, представления, проектирования и документирования систем. UML содержит стандартный набор диаграмм: вариантов использования, классов, поведения, взаимодействия, состояний, деятельностей, реализации.

Преимущества объектно-ориентированного подхода:

- Объектная декомпозиция даёт возможность создавать модели меньшего размера путём использования общих механизмов.
- Позволяет избежать создания сложных моделей, так как предполагает эволюционный путь развития на базе относительно небольших подсистем.
- Объектная модель естественна, поскольку ориентирована на человеческое восприятие мира.

Недостатки:

- Высокие начальные затраты. Подход не даёт немедленной отдачи.
- Эффект сказывается после разработки двух-трёх проектов и накопления повторно используемых компонентов.
- Диаграммы менее наглядны для заказчика.

Сравнение методик моделирования

В функциональных моделях главными структурными компонентами являются функции, которые связываются между собой потоками объектов.

Достоинства функциональных моделей:

- Реализация структурного подхода «сверху-вниз».
- Модульное проектирование ИС.
- Процедурная строгость декомпозиции.
- Наглядность представления.

Главный недостаток: процессы и данные существуют отдельно друг от друга. Помимо функциональной декомпозиции существует структура данных, находящаяся на втором плане. Кроме того, не ясны условия выполнения процессов обработки информации, которые динамически могут изменяться.

В объектно-ориентированных моделях главный структурообразующий компонент — класс объектов с набором функций. Для классов характерна иерархия обобщения, позволяющая осуществлять наследование атрибутов и функций.

При объектно-ориентированном подходе сначала выделяются классы объектов, а затем в зависимости от возможных состояний объектов определяются методы обработки. Это обеспечивает наилучшую реализацию динамического поведения системы.

Критерий выбора. Для более регламентированных задач больше подходят функциональные модели. Для более адаптивных бизнес-процессов (управление рабочими потоками, реализация динамических запросов к информационным хранилищам) — объектно-ориентированные. В рамках одной и той же ИС для различных классов задач могут требоваться различные виды моделей. В таком случае должны использоваться комбинированные модели.

Синтетическая методика

Каждая из рассмотренных методик позволяет решить задачу построения формального описания рабочих процедур исследуемой системы. Все методики позволяют построить модель «как есть» и «как должно быть». С другой стороны, каждая из них обладает существенными недостатками. Их можно суммировать так: недостатки применения отдельной методики лежат не в области описания реальных процессов, а в неполноте методического подхода.

Наилучшим способом преодоления недостатков является формирование **синергетической (синтетической) методики**, объединяющей различные этапы отдельных методик. Из каждой методики берётся часть, наиболее полно и

формально изложенная, и обеспечивается возможность обмена результатами на различных этапах.

Идея синтетической методики заключается в последовательном применении функционального и объектного подхода с учётом возможности реинжиниринга существующей ситуации.

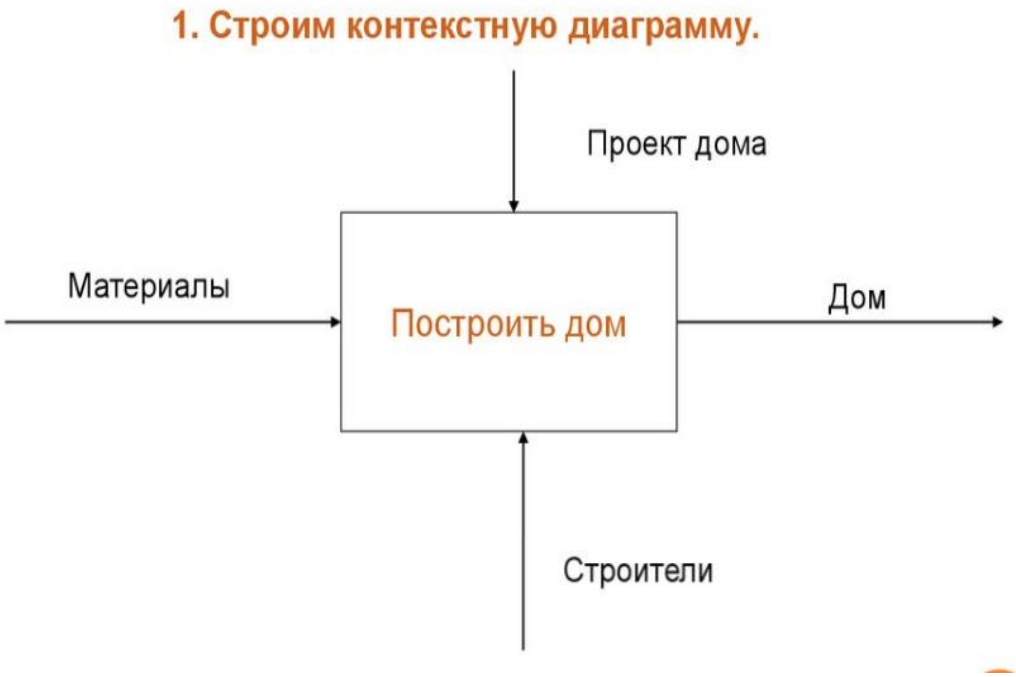
Рассмотрим применение синтетической методики на примере разработки административного регламента. Выделяются следующие стадии:

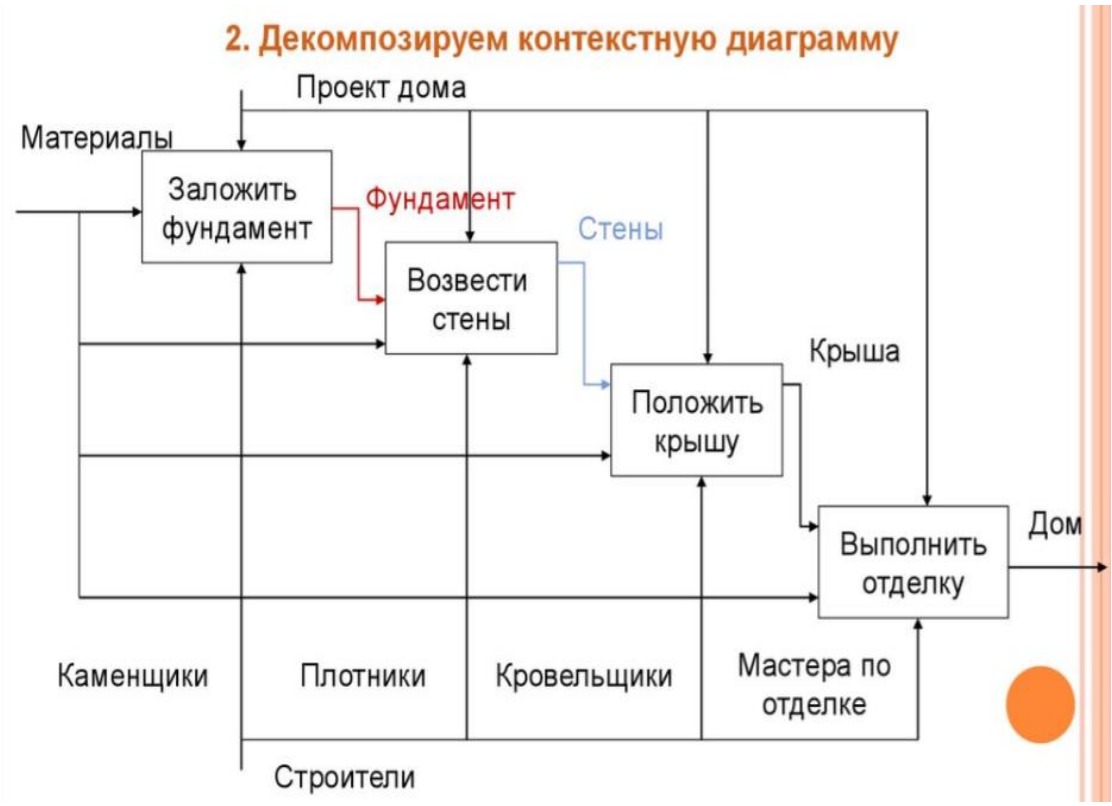
1. **Определение границ системы.** При помощи анализа потоков данных выделяют внешние сущности и собственно моделируемую систему.
2. **Выделение сценариев использования системы.** При помощи критерия полезности строят для каждой внешней сущности набор сценариев использования системы.
3. **Добавление системных сценариев использования.** Определяют сценарии, необходимые для реализации целей системы, отличных от целей пользователей.
4. **Построение диаграммы активностей по сценариям использования.** Строят набор действий системы, приводящих к реализации сценариев.
5. **Функциональная декомпозиция диаграмм активностей** как контекстных диаграмм методики IDEF0.
6. **Формальное описание отдельных функциональных активностей** в виде административного регламента.

Пример. Для кадрового агентства синтетическая методика может выглядеть так:

1. Определяем границы: система — «ИС учёта безработных и вакансий», внешние сущности — соискатели, предприятия, налоговые органы.
2. Выделяем сценарии для соискателя: «Подать резюме», «Найти вакансию», «Получить направление».
3. Добавляем системные сценарии: «Сформировать отчёт для налоговой», «Обновить базу вакансий».
4. Строим диаграммы активностей по каждому сценарию.
5. Декомпозируем активности в IDEF0.
6. Описываем регламент работы системы.

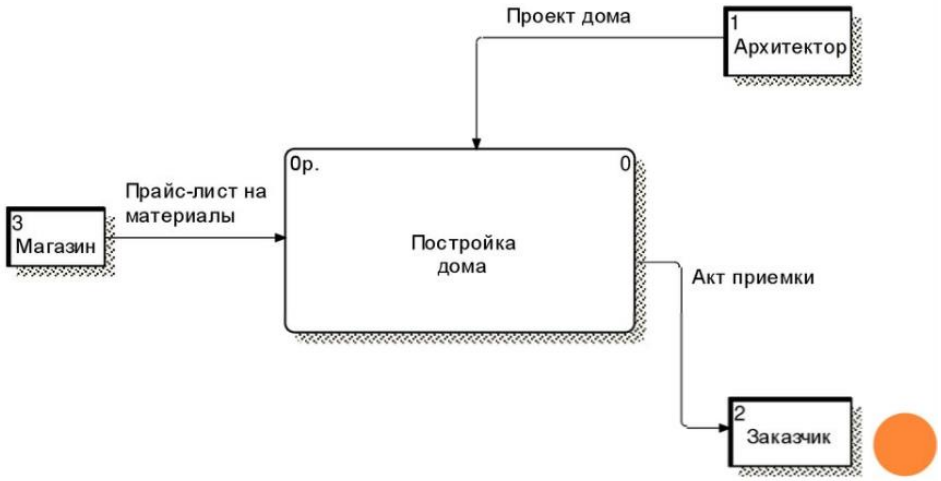
Пример модели процесса постройки дома



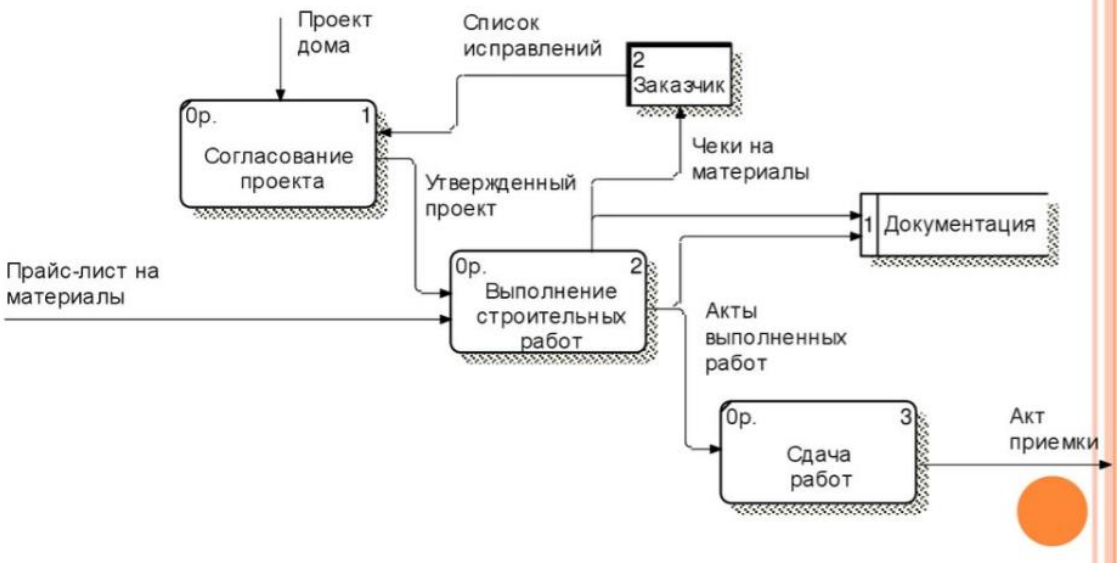


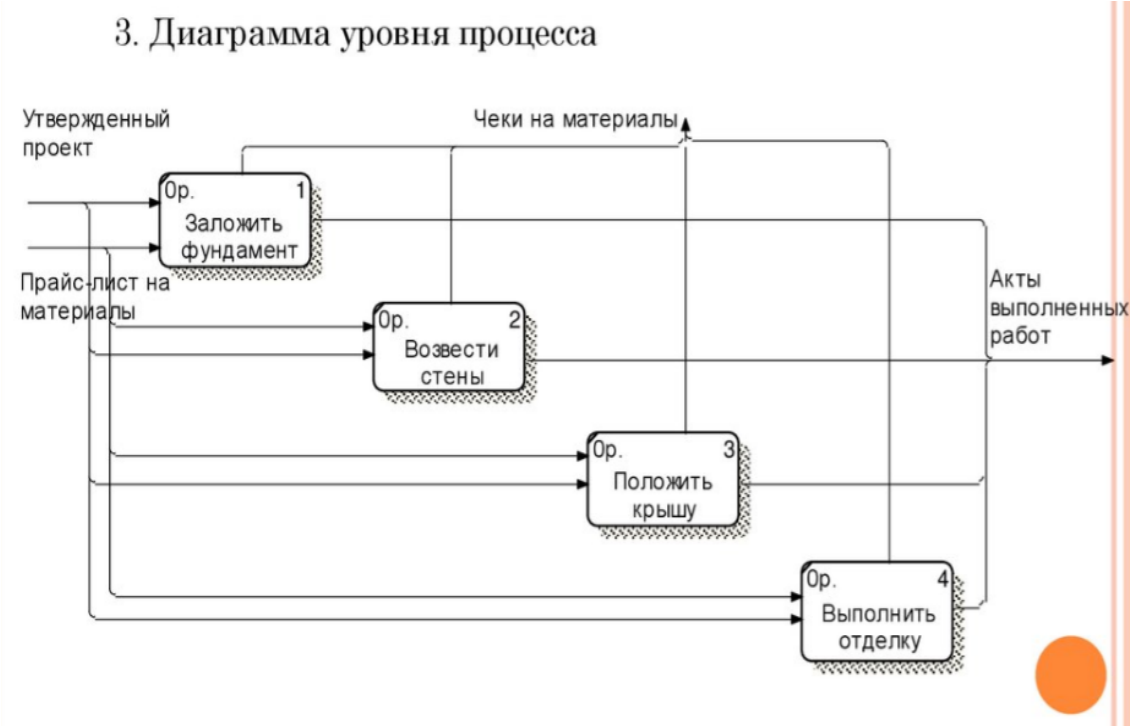
Пример DFD модели постройки дачного дома

1. Контекстная диаграмма уровня системы



2. Диаграмма уровня подсистемы





Итог

- **Модель предметной области** — это система, имитирующая структуру или функционирование реальной области. Без неё проектирование ИС превращается в угадывание.
- **Структурный аспект** включает объектную, функциональную, управленческую, организационную и техническую структуры.
- **Три уровня детализации:** внешний (что?), концептуальный (как?), внутренний (чем?).
- **Функциональные методологии** (IDEF0, DFD) описывают систему как набор функций. Хороши для регламентированных задач, но плохо показывают динамику.
- **Объектно-ориентированная методология** (UML) описывает систему как набор объектов. Лучше подходит для адаптивных процессов, но требует высоких начальных затрат.
- **Синтетическая методика** объединяет достоинства обоих подходов: сначала функциональное описание, затем объектное, с учётом реинжиниринга.

Моделирование предметной области — это фундамент, на котором строится вся информационная система. Чем качественнее построена модель, тем меньше ошибок будет при разработке и тем дешевле обойдётся сопровождение системы в будущем.